

REDROB AI · DATA & AI CHALLENGE

Intelligent Candidate Ranking System

Ranking candidates the way a **great recruiter would** — not by matching keywords, but by understanding who genuinely fits the role.

MULTI-SIGNAL HYBRID SCORING

PRODUCTION-FIRST FILTERING

100K CANDIDATES · 55 SECONDS

THE PROBLEM

Why keyword filters **fail**



Shallow Matching

A candidate who listed "Machine Learning" gets the same score whether they trained academic models or shipped ML to 10 million users.



Skills List $\neq$ Reality

Career history often reveals deeper expertise than the skills section. "Deployed embedding-based retrieval to production" won't appear as a keyword.



Missing Disqualifiers

The JD explicitly disqualifies pure-research profiles and non-coding managers. Keyword search can't detect these patterns.

Our answer: Read the full picture — skills, career narratives, behavioral signals, platform activity — and score like a senior technical recruiter.

JOB DESCRIPTION ANALYSIS

What Redrob AI **actually** needs

✓ Hard Requirements (Disqualify if missing)

- ▶ Production embeddings-based retrieval (sentence-transformers, BGE, E5, OpenAI)
- ▶ Vector DB / hybrid search (Milvus, FAISS, Pinecone, Weaviate, Qdrant, Elasticsearch)
- ▶ Strong Python — code quality matters
- ▶ Ranking evaluation frameworks (NDCG, MRR, MAP, A/B testing)
- ▶ 5–9 years (range, not strict — judgment matters)

★ Nice-to-Have (differentiate top candidates)

LLM FINE-TUNING (LORA/QLORA)

LEARNING-TO-RANK

HR-TECH / MARKETPLACE EXPERIENCE

DISTRIBUTED SYSTEMS AT SCALE

OPEN-SOURCE ML CONTRIBUTIONS

✗ Explicit Disqualifiers (stated in JD)

- ✗ Pure research environment — no production deployment ever
- ✗ "AI experience" = only LangChain wrappers (<12 months)
- ✗ No production code in last 18 months (architecture-only roles)
- ✗ Clearly non-tech roles (HR Manager, Operations, Sales)
- ✗ Very junior (<2 years) or stale profiles (inactive >1 year)

SYSTEM ARCHITECTURE

Multi-Signal Hybrid Scoring Pipeline



INPUT: CANDIDATES.JSONL → STREAM-PROCESSED, NO FULL-FILE RAM LOAD

```
# Streaming with min-heap – handles 487MB without memory spike
with open("candidates.jsonl") as f:
    for line in f:
        candidate = json.loads(line)
        score, breakdown = compute_composite_score(candidate)
        if len(heap) < 100:
            heapq.heappush(heap, (score, cid, breakdown, candidate))
        elif score > heap[0][0]:
            heapq.heapreplace(heap, (score, cid, breakdown, candidate))
```

SCORING ENGINE

6 Signals, Weighted Composite

 Core AI/ML Skills Match	35%		Proficiency × tier weight × duration × log(endorsements) + assessment scores
 Experience Fit	20%		Gaussian peak at 7 yrs ($\sigma=3$) + title relevance + industry bonus
 Career Depth / Production	15%		Production keywords in descriptions vs. research-only language
 Platform Behavioral Signals	15%		Response rate, GitHub activity, interview completion, verification, recency
 Nice-to-Have Bonuses	10%		LoRA/fine-tuning, LTR, HR-tech experience, OSS contributions, tier-1 edu
 Disqualifier Penalties	×(1-p)		Multiplicative penalty: pure research, no AI skills, non-tech title, inactive



Formula: $\text{final_score} = (\sum \text{weight}_i \times \text{signal}_i) \times (1 - \text{penalty})$

SIGNAL 1 DEEP-DIVE

3-Tier Skills Taxonomy

Tier 1 — Must Have

3.0×

- ✦ Sentence Transformers / BGE / E5
- ✦ FAISS / Milvus / Qdrant / Pinecone
- ✦ Elasticsearch / OpenSearch / pgvector
- ✦ BM25 / Hybrid Search / Dense Retrieval
- ✦ NDCG / MRR / Retrieval Evaluation
- ✦ MLOps / Model Serving / Production ML
- ✦ Python (production-grade)

Tier 2 — Nice to Have

2.0×

- ◆ LoRA / QLoRA / PEFT / RLHF
- ◆ LangChain / LlamaIndex / RAG
- ◆ PyTorch / TensorFlow / JAX
- ◆ XGBoost / LightGBM (for LTR)
- ◆ MLflow / W&B / BentoML / Ray
- ◆ Spark / Kafka / Kubernetes
- ◆ NLP / ASR / TTS / Transformers

Tier 3 — General Eng

0.5×

- ◆ SQL / Pandas / NumPy
- ◆ React / JavaScript / TypeScript
- ◆ Docker / Git / Linux
- ◆ Image Classification / Object Detection
- ◆ Excel / PowerPoint / Tableau
- ◆ Project Management / Agile
- ◆ Marketing / Accounting / HR

Scoring per skill: $\text{tier_weight} \times \text{proficiency_weight} \times \min(1, \text{duration_months}/24) \times \log(1+\text{endorsements}) / \log(51)$

Also scans career history *text* for keyword evidence — catches expertise not listed in skills section.

Production vs. Research Detection

The JD's biggest disqualifier: *"Pure research environments without production deployment."* We detect this by scanning career description text.

✓ Production Signal Keywords

DEPLOYED PRODUCTION REAL USERS AT SCALE
SERVING INFERENCE A/B TEST RETRIEVAL SYSTEM
ROLLOUT MONITORING LATENCY THROUGHPUT
MILLIONS OF INDEX REFRESH NDCG / MRR

✗ Research-Only Signal Keywords

RESEARCH PAPER ARXIV PUBLICATION JOURNAL
THESIS ACADEMIC POSTDOC UNIVERSITY LAB
BENCHMARK DATASET CITATION LITERATURE REVIEW

Example: HIGH production score

"Deployed embedding-based retrieval to production serving 5M daily requests. Set up A/B testing framework with NDCG metric. Handled embedding drift and index refresh cycles."

Example: PENALIZED profile

"Published 3 papers on transformer architectures. Thesis on few-shot learning. Research internship at university lab. Benchmark evaluation on standard datasets."

100K
CANDIDATES
PROCESSED

55s
TOTAL
RUNTIME

CANDIDATES
PROCESSED

TOTAL
RUNTIME

TOP SCORE



TITLE DISTRIBUTION (TOP 100)

Applied ML Engineer	15
Search Engineer	12
ML Engineer	12
AI Engineer	11
Sr. Data Scientist	10

NLP Engineer  8

Others (Rec.Sys, Staff ML)

  32

Why This Architecture?

No API Required — Fully Local

Deterministic, reproducible, zero cost. No rate limits, no latency from external services. Processes 100K candidates in 55 seconds on CPU alone.

Semantic Understanding

Rather than embedding the JD and using cosine similarity (which requires an API), we extract the semantics manually as a structured skill taxonomy — more interpretable and debuggable.

Career Narrative Analysis

We read job description text, not just skill tags. "Deployed to 5M users" reveals more than a "Machine Learning" checkbox. 20+ production keywords, 11 research-penalty keywords scanned per role.

Explainable Scores

Every score is decomposed into 6 named signals. A recruiter can look at any candidate and understand exactly why they ranked where they did — not a black-box embedding distance.



Behavioral Signals as Quality Filter

Platform engagement (response rate, GitHub activity, interview completion) differentiates equally-skilled candidates. A candidate with 0.95 response rate > 0.05, all else equal.



$O(n)$ Memory — Min-Heap Design

The 487MB file is streamed line-by-line. A min-heap of size 100 holds only the current top candidates. Space complexity is $O(100)$ regardless of input size.

SUMMARY

What We **Delivered**



GitHub Repo

Clean, documented, production-quality Python code. `skills_taxonomy.py` + `score_signals.py` + `ranker.py`

[GITHUB.COM/TECH_FUSION/INDIA-RUNS](https://github.com/tech_fusion/india-runs)



submission.csv

Top 100 ranked candidates, validated. 100% AI/ML professionals, non-increasing scores, unique IDs.

✓ SUBMISSION IS VALID.



This Deck

Architecture, scoring logic, JD analysis, results breakdown — the full story of how and why.

PRINT → SAVE AS PDF

100,000 candidates ranked in **55 seconds**. A shortlist every recruiter can trust.